

ПРАКТИЧНА РОБОТА №4

Тема: ПОКАЖЧИКИ

Звіт про виконання практичної роботи

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Оволодіння практичними навичками роботи з покажчиками в мовах C/C++.

Теоретичні відомості

Покажчик - це спеціальна змінна, значенням якої є адреса іншої змінної або області пам'яті. За допомогою покажчика можна звертатися не лише до адреси, а й до значення, яке зберігається за цією адресою. Для цього використовується операція розіменування *.

Оголошення покажчика виконується із зазначенням типу даних, на які він буде вказувати, наприклад: `int* p;`, `char* pc;`, `double* pd;`. Для отримання адреси змінної застосовується оператор `&`, а для доступу до значення за адресою - оператор `*`.

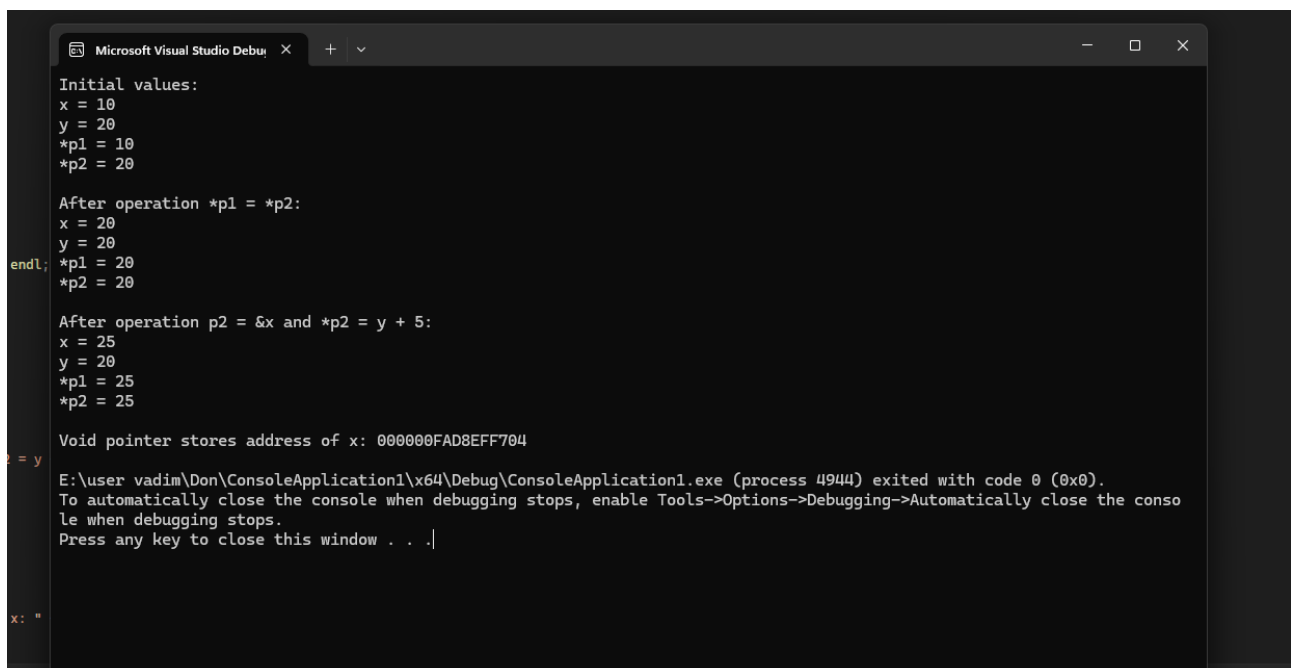
Покажчики використовуються під час роботи з масивами, динамічною пам'яттю, структурами даних і функціями. Вони дозволяють ефективно передавати дані, змінювати значення змінних через їх адреси, створювати динамічні масиви та працювати з елементами масивів за допомогою адресної арифметики.

Для динамічного виділення пам'яті в C++ використовуються оператори `new` та `delete`, а для масивів - `new[]` та `delete[]`. У мові C для цього застосовують функції `malloc()` та `free()`.

Хід роботи

Приклад 1. Базова робота з покажчиками

У першому прикладі розглянуто оголошення покажчиків, присвоєння їм адрес змінних, розіменування покажчиків і зміну значень змінних через покажчики. Також показано приклад використання покажчика типу `void*`, який може зберігати адресу об'єкта будь-якого типу.



```
Microsoft Visual Studio Debug Console
Initial values:
x = 10
y = 20
*p1 = 10
*p2 = 20

After operation *p1 = *p2:
x = 20
y = 20
*p1 = 20
*p2 = 20

After operation p2 = &x and *p2 = y + 5:
x = 25
y = 20
*p1 = 25
*p2 = 25

Void pointer stores address of x: 000000FAD8EFF704

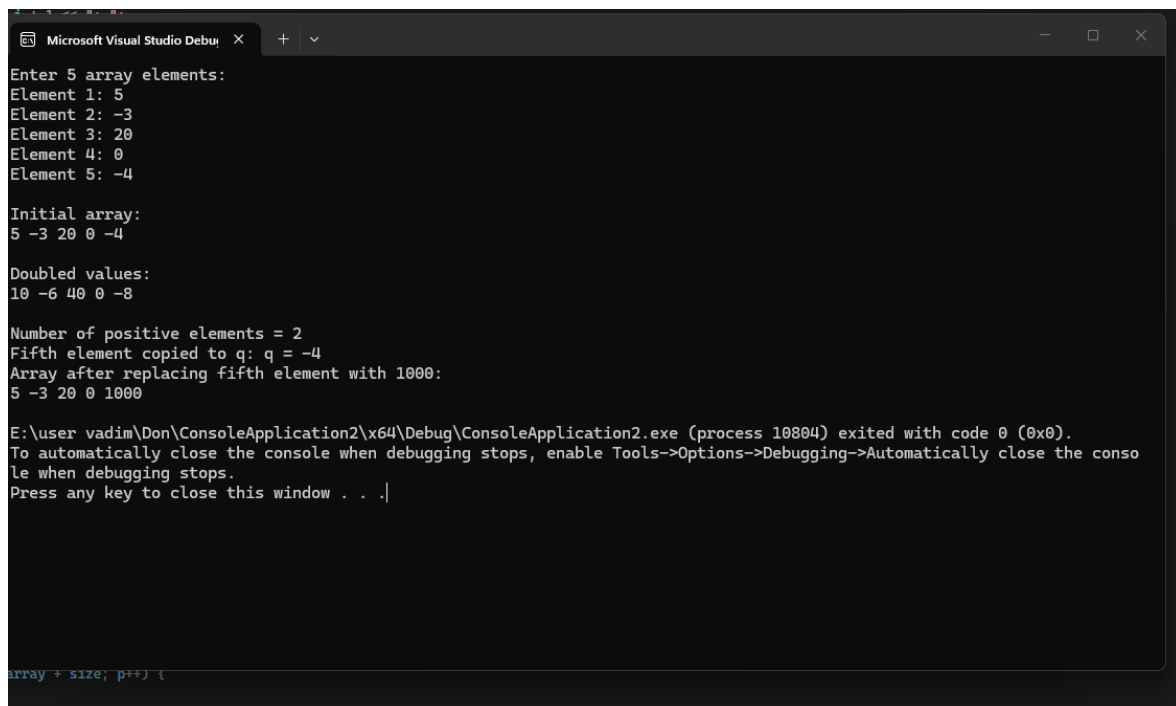
E:\user vadin\Don\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 4944) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Рисунок 1 - Результат базової роботи з покажчиками

Приклад 2. Покажчики та одновимірний динамічний масив

У другому прикладі створено одновимірний динамічний масив. Елементи масиву вводяться з клавіатури та обробляються за допомогою покажчиків. Програма

виводить початковий масив, формує подвоєні значення, підраховує кількість додатних елементів, копіює п'ятий елемент у змінну та замінює його значенням 1000.



```
Microsoft Visual Studio Debug
Enter 5 array elements:
Element 1: 5
Element 2: -3
Element 3: 20
Element 4: 0
Element 5: -4

Initial array:
5 -3 20 0 -4

Doubled values:
10 -6 40 0 -8

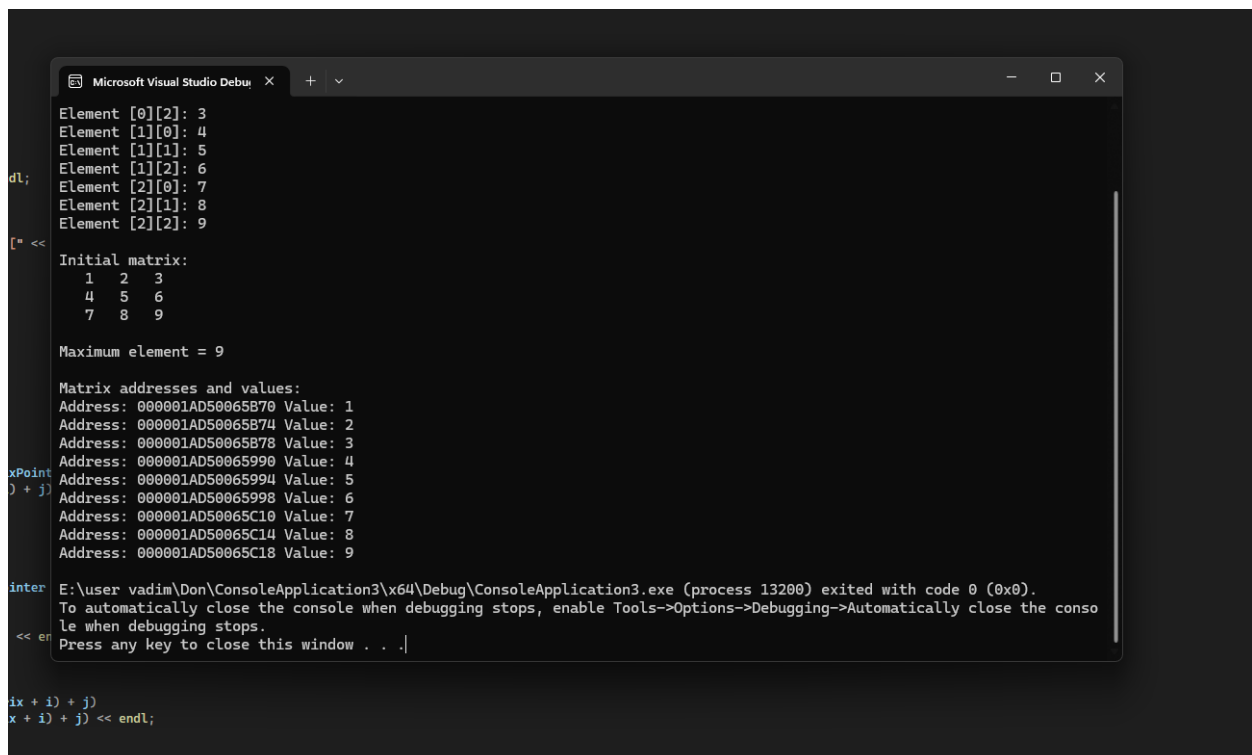
Number of positive elements = 2
Fifth element copied to q: q = -4
Array after replacing fifth element with 1000:
5 -3 20 0 1000

E:\user vadim\Don\ConsoleApplication2\x64\Debug\ConsoleApplication2.exe (process 10804) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 2 - Результат роботи з одновимірним динамічним масивом через покажчик

Приклад 3. Покажчики та двовимірний динамічний масив

У третьому прикладі створено двовимірний динамічний масив за допомогою покажчика на покажчик. Доступ до елементів матриці здійснюється через адресний вираз $*(matrix + i) + j$. Програма виводить матрицю, знаходить максимальний елемент, а також демонструє адреси та значення елементів.



```
Microsoft Visual Studio Debug
Element [0][2]: 3
Element [1][0]: 4
Element [1][1]: 5
Element [1][2]: 6
Element [2][0]: 7
Element [2][1]: 8
Element [2][2]: 9

Initial matrix:
1 2 3
4 5 6
7 8 9

Maximum element = 9

Matrix addresses and values:
Address: 000001AD50065B70 Value: 1
Address: 000001AD50065B74 Value: 2
Address: 000001AD50065B78 Value: 3
Address: 000001AD50065990 Value: 4
Address: 000001AD50065994 Value: 5
Address: 000001AD50065998 Value: 6
Address: 000001AD50065C10 Value: 7
Address: 000001AD50065C14 Value: 8
Address: 000001AD50065C18 Value: 9

E:\user vadim\Don\ConsoleApplication3\x64\Debug\ConsoleApplication3.exe (process 13200) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 3 - Результат роботи з двовимірним динамічним масивом через покажчики

Контрольні запитання та відповіді

1. Що таке покажчик? Як виконується оголошення покажчиків?

Покажчик - це змінна, яка зберігає адресу іншої змінної або області пам'яті. Оголошення покажчика виконується із зазначенням типу даних, на які він буде вказувати. Наприклад, `int* p;` оголошує покажчик на змінну типу `int`, `double* pd;` - покажчик на змінну типу `double`. Для присвоєння адреси використовується оператор `&`: `p = &x;`.

2. Як за допомогою покажчика можна звернутися до вмісту змінної, на яку він вказує?

Щоб звернутися до вмісту змінної, на яку вказує покажчик, використовується операція розіменування `*`. Якщо `p = &x;`, то вираз `*p` означає значення змінної `x`. Через `*p` можна не тільки прочитати значення, а й змінити його, наприклад: `*p = 25;`.

3. Які арифметичні операції можна виконувати над покажчиками? Навести приклади.

Над покажчиками можна виконувати додавання або віднімання цілого числа, інкремент і декремент, порівняння покажчиків, а також віднімання одного покажчика від іншого, якщо вони вказують на елементи одного масиву. Наприклад, якщо `p` вказує на перший елемент масиву, то `p + 1` вказує на наступний елемент, а `*(p + i)` дає значення `i`-го елемента.

4. Що таке покажчик типу void?

Покажчик типу `void*` - це універсальний покажчик, який може зберігати адресу об'єкта будь-якого типу. Наприклад, у `void* ptr;` можна записати адресу змінної типу `int`, `float` або структури. Проте перед розіменуванням такий покажчик потрібно привести до конкретного типу.

5. Як звернутися до елемента одновимірного масиву через покажчик?

Якщо ім'я масиву або покажчик вказує на перший елемент, то до елемента з індексом `i` можна звернутися через вираз `*(array + i)`. Наприклад, `*(A + 2)` відповідає елементу `A[2]`. Також можна використовувати запис `p[i]`, якщо `p` є покажчиком на початок масиву.

6. Як звернутися до елемента двовимірного масиву через покажчик?

До елемента двовимірного масиву через покажчик можна звернутися за допомогою подвійного розіменування. Для динамічного масиву `matrix` елемент з рядка `i` та стовпця `j` можна отримати виразом `*(*(matrix + i) + j)`. Цей запис еквівалентний `matrix[i][j]`.

7. Які є функції виділення та звільнення пам'яті під масив? Навести приклад.

У C для динамічного виділення пам'яті застосовують `malloc()`, а для звільнення - `free()`. У C++ використовують оператори `new` і `delete`, а для масивів - `new[]` і `delete[]`. Наприклад: `int* A = new int[N];` виділяє пам'ять під масив з `N` цілих чисел, а `delete[] A;` звільняє цю пам'ять.

8. Чим відрізняється розміщення в пам'яті одновимірного та двовимірного масивів?

Одновимірний масив розміщується як послідовна ділянка пам'яті, де елементи йдуть один за одним. Двовимірний масив можна розглядати як таблицю. Статичний двовимірний масив зазвичай зберігається послідовно рядками, а динамічний двовимірний масив часто створюється як масив покажчиків на окремі рядки. У такому випадку рядки можуть розміщуватися в різних ділянках пам'яті.

Висновок

У ході виконання практичної роботи було розглянуто основні принципи роботи з покажчиками в C++. Було досліджено оголошення покажчиків, отримання адрес змінних, розіменування, використання покажчика типу `void*`, а також роботу з одновимірними та двовимірними динамічними масивами.

Практичні приклади показали, що покажчики є важливим інструментом для роботи з пам'яттю, масивами та адресною арифметикою. Вони дозволяють ефективно обробляти дані, створювати динамічні структури та керувати пам'яттю під час виконання програми.